

TestPlatform

Rapport d'Execution de Campagne

ms_gestion

Projet: ms_gestion

Version: 2.5

Environnement: ms_gestion_env

Debut: 17/06/2026 09:35 | Fin: 17/06/2026 09:42

Duree: 389.9 s

Rapport genere le 17/06/2026 09:42

ECHEC

Resume executif

Total	Succes	Echecs	Erreurs	Duree
3	0	3	0	389.9 s

Taux de reussite



Evolution vs campagne precedente: DOWN 50.0 points

Campagne precedente #20

Corrections IA: 1 test(s) corrige(s) automatiquement, 0 avec succes

Informations du contexte

Projet	ms_gestion
Description	test

Suites

Nom	Type	Depot Git	Branche
Suite UNIT	UNIT	N/A	main

Cas de test

Titre	Type	Priorite	Risque	Script
test case add	UNIT	3	CRITICAL	IA
test add () suite test	UNIT	1	CRITICAL	IA
project create	UNIT	1	HIGH	IA

Environnement

URL Web	N/A
URL API	N/A
Repository	https://github.com/ilyeschaabani/Automatisation_intelligente_des_tests_applicatifs
Branche	main
Base de donnees	POSTGRESQL

Resultats details

Synthese par suite

Suite	Total	Succes	Echecs	Taux
Suite UNIT	3	0	3	0%

Suite UNIT

test case add			FAILURE	
Type	UNIT	Duree	9.1 s	
Script	IA	Erreur	[COMPILATION_ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.827 s <<< FAILURE! -- in suites.unit.TestCaseServiceTest	
test add () suite test			IA x1	FAILURE
Type	UNIT	Duree	12.4 s	
Script	IA	Erreur	[COMPILATION_ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.563 s <<< FAILURE! -- in suites.unit.TestSuiteServiceUnitTest	
project create			FAILURE	
Type	UNIT	Duree	7.1 s	
Script	IA	Erreur	[COMPILATION_ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.513 s <<< FAILURE! -- in suites.unit.ProjectServiceTest	

Analyse IA

test case add

Erreurs extraites	<pre>[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.827 s <<< FAILURE! -- in suites.unit.TestCaseServiceTest [ERROR] suites.unit.TestCaseServiceTest.testAddTestCase -- Time elapsed: 0.052 s <<< FAILURE! [ERROR] Failures: [ERROR] TestCaseServiceTest.testAddTestCase:101 [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire- plugin:3.2.5:test (default-test) on project ms_gestion: There are test failures. [ERROR] [ERROR] Please refer to C:\tmp\ms-execution\exec- 13176559556184699839\Backend\ms_gestion\target\surefire-reports for the individual test results. [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream. [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
Analyse IA	**Cause** : La cause de l'erreur est une référence à une méthode statique qui n'a pas pu être mockée correctement par Mockito. Plus précisément, la méthode <code>`SecurityUtils.getCurrentUserId()`</code> n'est pas mockée correctement dans le test <code>`TestCaseServiceTest.testAddTestCase`</code>. **Correction** : Pour corriger l'erreur de compilation, vous devriez ajouter une configuration d'ouverture des mocks à votre test comme suit: <pre>```java @BeforeMethod public void setUp() { MockitoAnnotations.openMocks(this); } ```</pre> Vous pouvez également utiliser l'API <code>`Mockito.mockStatic()`</code> pour mock une classe statique comme suit: <pre>```java private MockedStatic<SecurityUtils> mockedSecurity; @BeforeMethod public void setUp() { mocks = MockitoAnnotations.openMocks(this); mockedSecurity = Mockito.mockStatic(SecurityUtils.class); mockedSecurity.when(() -> SecurityUtils.getCurrentUserId()).thenReturn(1L); } ```</pre> Cela devrait résoudre l'erreur de compilation. **Conseil** : Assurez-vous toujours que toutes vos méthodes statiques sont correctement mockées pour éviter ce type d'erreurs de compilation dans vos tests unitaires.

test add () suite test

Erreurs extraites	<pre>[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.563 s <<< FAILURE! -- in suites.unit.TestSuiteServiceUnitTest [ERROR] suites.unit.TestSuiteServiceUnitTest.test_add_happyPath -- Time elapsed: 0.034 s <<< FAILURE! java.lang.AssertionError: expected [TestProject] but found [null] [ERROR] Failures: [ERROR] TestSuiteServiceUnitTest.test_add_happyPath:89 expected [TestProject] but found [null] [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire- plugin:3.2.5:test (default-test) on project ms_gestion: There are test failures. [ERROR] [ERROR] Please refer to C:\tmp\ms-execution\exec- 13176559556184699839\Backend\ms_gestion\target\surefire-reports for the individual test results. [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream. [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
Analyse IA	**Cause** : Lors de l'exécution du test, `testSuiteService.add()` retourne une réponse null. Cela signifie que le méthode `save()` dans `TestSuiteRepository` n'est pas appelée correctement et ne renvoie pas de données TestSuite valides. **Correction** : La cause probable est l'ordre des instructions dans `setUp()` et la présence d'instructions qui devraient être écrites avant le mocking des réponses du serveur. Dans cette situation, après avoir appelé `MockitoAnnotations.openMocks(this)`, vous devez initialiser les champs avec des valeurs prédéfinies pour que votre test soit exécuté sans erreur. Vérifiez si le constructeur de TestSuite est correctement configuré et renvoie une instance valide. **Conseil** : Assurez-vous d'appeler `MockitoAnnotations.openMocks(this)` avant l'instanciation des classes avec les annotations `@Mock` ou `@InjectMocks` pour que Mockito puisse gérer leurs interactions et renseigner les champs requis par votre classe de test.

project create

Erreurs extraites	<pre>[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.513 s <<< FAILURE! -- in suites.unit.ProjectServiceTest [ERROR] suites.unit.ProjectServiceTest.test_create_happy_path -- Time elapsed: 0.036 s <<< FAILURE! [ERROR] Failures: [ERROR] ProjectServiceTest.test_create_happy_path:44 Runtime Utilisateur non authentifi [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire- plugin:3.2.5:test (default-test) on project ms_gestion: There are test failures. [ERROR] [ERROR] Please refer to C:\tmp\ms-execution\exec- 13176559556184699839\Backend\ms_gestion\target\surefire-reports for the individual test results. [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream. [ERROR] -> [Help 1]</pre>
------------------------------	---

```
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the
following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
```

Analyse
IA

Réponse

Cause

Le test ne compile pas car la méthode `create` de l'objet `ProjectService` n'est pas définie. La classe `ProjectServiceTest` tente d'appeler cette méthode, mais elle n'a pas été correctement initialisée avec les objets mockés.

Correction

Pour résoudre ce problème, il faut ajouter la déclaration de la méthode `create` dans la classe `ProjectService` pour qu'elle puisse être appelée par le test. Cela ressemble à ça:

```
```java
public class ProjectService {
// Autres méthodes et attributs...

public ProjectResponse create(CreateProjectRequest request) {
// Logique de création d'un projet...
}
}
```
```

Assurez-vous que la signature de la méthode `create` correspond exactement à celle définie dans le test. Dans votre cas, elle devrait être:

```
```java
public ProjectResponse create(CreateProjectRequest request) { ... }
```
```

et non:

```
```java
private ProjectResponse create(CreateProjectRequest request) { ... } // Erreur de
compilation car la méthode n'est pas accessible depuis l'extérieur de la classe.
```
```

Conseil

En Java, toute méthode doit avoir une déclaration publique ou protégée (public or protected) pour qu'elle puisse être appelée par d'autres classes. Un mot clé `private` rend la méthode accessible uniquement dans sa propre classe, ce qui est contraire à l'intention générale de programmation orientée objet. Assurez-vous que les méthodes que vous testez sont publiques ou protégées.

Corrections automatiques par IA

L'IA a detecte des erreurs dans certains scripts generes et les a corriges automatiquement.

test add () suite test

1 correction(s)

FAILURE

```
=== Tentative 1/3 ===
Erreur détectée: [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.563 s <<<
FAILURE! -- in suites.unit.TestSuiteServiceUnitTest
[ERROR] suites.unit.TestSuiteServiceUnitTest.test_add_happyPath -- Time elapsed: 0.034 s <<< FAILURE!
java.lang.AssertionError: expected [TestProject] but found [null]
[ERROR] Failures:
[ERROR]   TestSuiteServiceUnitTest.test_add_happyPath:89 expected [TestProject] but found [null]
[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
[E
Résultat: L'IA n'a pas pu corriger le script.
```

Metriques

Top 5 des tests les plus lents

| Test | Suite | Duree |
|------------------------|------------|--------|
| test add () suite test | Suite UNIT | 12.4 s |
| test case add | Suite UNIT | 9.1 s |
| project create | Suite UNIT | 7.1 s |

Temps d'execution par type

| Type | Duree moyenne |
|------|---------------|
| UNIT | 9.5 s |

Tests instables

Aucun test instable detecte

Recommandations

- test case add:

****Cause**** : La cause de l'erreur est une référence à une méthode statique qui n'a pas pu être mockée correctement par Mockito. Plus précisément, la méthode `SecurityUtils.getCurrentUserId()` n'est pas mockée correctement dans le test `TestCaseServiceTest.testAddTestCase``.

****Correction**** : Pour corriger l'erreur de compilation, vous devriez ajouter une configuration d'ouverture des mocks à votre test comme suit:

```
```java
@BeforeMethod
public void setUp() {
 MockitoAnnotations.openMocks(this);
}
```
```

Vous pouvez également utiliser l'API `Mockito.mockStatic()` pour mock une classe statique comme suit:

```
```java
private MockedStatic<SecurityUtils> mockedSecurity;

@BeforeMethod
public void setUp() {
 mocks = MockitoAnnotations.openMocks(this);
 mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
 mockedSecurity.when(() -> SecurityUtils.getCurrentUserId()).thenReturn(1L);
}
```
```

Cela devrait résoudre l'erreur de compilation.

****Conseil**** : Assurez-vous toujours que toutes vos méthodes statiques sont correctement mockées pour éviter ce type d'erreurs de compilation dans vos tests unitaires.

- test add () suite test:

****Cause**** : Lors de l'exécution du test, `testSuiteService.add()` retourne une réponse null. Cela signifie que la méthode `save()` dans `TestSuiteRepository`` n'est pas appelée correctement et ne renvoie pas de données TestSuite valides.

****Correction**** : La cause probable est l'ordre des instructions dans `setUp()` et la présence d'instructions qui devraient être écrites avant le mocking des réponses du serveur. Dans cette situation, après avoir appelé `MockitoAnnotations.openMocks(this)``, vous devez initialiser les champs avec des valeurs prédéfinies pour que votre test soit exécuté sans erreur. Vérifiez si le constructeur de TestSuite est correctement configuré et renvoie une instance valide.

****Conseil**** : Assurez-vous d'appeler `MockitoAnnotations.openMocks(this)`` avant l'instanciation des classes avec les annotations `@Mock`` ou `@InjectMocks`` pour que Mockito puisse gérer leurs interactions et renseigner les champs requis par votre classe de test.

- project create:

Réponse

Cause

Le test ne compile pas car la méthode `create` de l'objet `ProjectService` n'est pas définie. La classe `ProjectServiceTest` tente d'appeler cette méthode, mais elle n'a pas été correctement initialisée avec les objets mockés.

Correction

Pour résoudre ce problème, il faut ajouter la déclaration de la méthode `create` dans la classe `ProjectService` pour qu'elle puisse être appelée par le test. Cela ressemble à ça:

```
```java
public class ProjectService {
// Autres méthodes et attributs...

public ProjectResponse create(CreateProjectRequest request) {
// Logique de création d'un projet...
}
}
```
```

Assurez-vous que la signature de la méthode `create` correspond exactement à celle définie dans le test. Dans votre cas, elle devrait être:

```
```java
public ProjectResponse create(CreateProjectRequest request) { ... }
```
```

et non:

```
```java
private ProjectResponse create(CreateProjectRequest request) { ... } // Erreur de compilation car la méthode n'est pas
accessible depuis l'extérieur de la classe.
```
```

Conseil

En Java, toute méthode doit avoir une déclaration publique ou protégée (public or protected) pour qu'elle puisse être appelée par d'autres classes. Un mot clé `private` rend la méthode accessible uniquement dans sa propre classe, ce qui est contraire à l'intention générale de programmation orientée objet. Assurez-vous que les méthodes que vous testez sont publiques ou protégées.

Annexe - Scripts generes par IA

test case add

```
package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestCaseRequest;
import com.pfe.platform.ms_gestion.dto.response.TestCaseResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestCase;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.TestCaseRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.LlmService;
import com.pfe.platform.ms_gestion.service.TestCaseService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
import java.util.Optional;

public class TestCaseServiceTest {

    @Mock private TestCaseRepository testCaseRepository;
    @Mock private TestSuiteRepository testSuiteRepository;
    @Mock private ProjectMemberRepository projectMemberRepository;
    @Mock private LlmService llmService;
    @InjectMocks private TestCaseService testCaseService;

    private AutoCloseable mocks;
    private MockedStatic<SecurityUtils> mockedSecurity;

    @BeforeMethod
    public void setUp() {
        mocks = MockitoAnnotations.openMocks(this);
        mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
        mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mockedSecurity.close();
        mocks.close();
    }

    @Test
    public void testAddTestCase() {
        Long suiteId = 1L;

        Project project = new Project();
        project.setId(10L);
        project.setName("Test Project");

        ProjectMember member = new ProjectMember();
        member.setRole(ProjectMember.Role.ADMIN);
        member.setUserId(1L);
        member.setProject(project);

        TestSuite suite = new TestSuite();
```

```

suite.setId(suiteId);
suite.setType(TestSuite.TestType.UNIT);
suite.setProject(project);

CreateTestCaseRequest request = new CreateTestCaseRequest();
request.setTitle("Login test");
request.setType("UNIT");
request.setRiskLevel("LOW");
request.setPriority(1);
request.setDescriptionAI("Test the add method");

Mockito.when(testSuiteRepository.findById(suiteId))
    .thenReturn(Optional.of(suite));
Mockito.when(projectMemberRepository.findByIdAndUserId(
    Mockito.any(), Mockito.eq(1L)))
    .thenReturn(Optional.of(member));

Mockito.when(llmService.generateTestCode(
    Mockito.any(), Mockito.any(), Mockito.any()))
    .thenReturn("public class FakeTest {}");

Mockito.when(testCaseRepository.save(Mockito.any()))
    .thenAnswer(inv -> {
        TestCase tc = inv.getArgument(0);
        tc.setSuite(suite);
        return tc;
    });

TestCaseResponse response = testCaseService.add(suiteId, request);

Assert.assertNotNull(response);
Assert.assertEquals(response.getTitle(), "Login test");
Assert.assertEquals(response.getRiskLevel(), "LOW");
Mockito.verify(testCaseRepository, Mockito.times(1)).save(Mockito.any());
// verify
Mockito.verify(llmService, Mockito.times(1))
    .generateTestCode(Mockito.any(), Mockito.any(), Mockito.any());
}
}

```

test add () suite test

```

package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestSuiteRequest;
import com.pfe.platform.ms_gestion.dto.response.TestSuiteResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.ProjectRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.TestSuiteService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

import java.util.Optional;

public class TestSuiteServiceUnitTest {

```

```
@Mock private TestSuiteRepository testSuiteRepository;
@Mock private ProjectRepository projectRepository;
@Mock private ProjectMemberRepository projectMemberRepository;

@InjectMocks
private TestSuiteService testSuiteService; // pas de new()

private AutoCloseable mocks;
private MockedStatic<SecurityUtils> mockedSecurity;

@BeforeMethod
public void setUp() {
    mocks = MockitoAnnotations.openMocks(this);
    mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
    mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
}

@AfterMethod
public void tearDown() throws Exception {
    mockedSecurity.close();
    mocks.close();
}

@Test
public void test_add_happyPath() {
    // Arrange
    Long projectId = 1L;

    Project project = new Project();
    project.setId(projectId);

    ProjectMember member = new ProjectMember();
    member.setUserId(1L);
    member.setRole(ProjectMember.Role.ADMIN);
    member.setProject(project);

    TestSuite saved = new TestSuite();
    saved.setId(10L);
    saved.setProject(project);

    // Mock repositories
    Mockito.when(projectRepository.findById(projectId))
        .thenReturn(Optional.of(project));
    Mockito.when(projectMemberRepository.findByProjectIdAndUserId(
        Mockito.eq(projectId), Mockito.eq(1L)))
        .thenReturn(Optional.of(member));
    Mockito.when(testSuiteRepository.save(Mockito.any()))
        .thenReturn(saved);

    // Request avec les test data générés
    CreateTestSuiteRequest request = new CreateTestSuiteRequest();
    request.setName("TestProject");
    request.setType("UNIT");
    request.setGitRepoUrl("https://github.com/testproject.git");
    request.setGitBranch("master");
    request.setModulePath("/src/main/java/com/example/test");

    // Act
    TestSuiteResponse response = testSuiteService.add(projectId, request);

    // Assert
    Assert.assertNotNull(response);
    Assert.assertEquals(response.getName(), "TestProject");
    Mockito.verify(testSuiteRepository, Mockito.times(1)).save(Mockito.any());
    Mockito.verify(projectRepository, Mockito.times(1)).findById(projectId);
}
}
```

project create

```
package suites.unit; //

import com.pfe.platform.ms_gestion.service.ProjectService;
import com.pfe.platform.ms_gestion.repository.ProjectRepository;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.dto.request.CreateProjectRequest;
import com.pfe.platform.ms_gestion.dto.response.ProjectResponse;
import org.mockito.*;
import org.testng.annotations.*;
import static org.testng.Assert.*;
import static org.mockito.Mockito.*;

public class ProjectServiceTest {

    private AutoCloseable mocks; //

    @Mock private ProjectRepository projectRepository;
    @Mock private ProjectMemberRepository projectMemberRepository;
    @InjectMocks private ProjectService sut;

    @BeforeMethod
    public void setUp() throws Exception {
        mocks = MockitoAnnotations.openMocks(this); //
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mocks.close(); //
    }

    @Test
    public void test_create_happy_path() {
        // given
        CreateProjectRequest request = new CreateProjectRequest();
        request.setName("test");

        Project saved = new Project();
        saved.setId(1L);
        saved.setName("test"); // entité réelle, pas null
        when(projectRepository.save(any())).thenReturn(saved);

        // when
        ProjectResponse result = sut.create(request);

        // then
        assertNotNull(result);
        assertEquals(result.getName(), "test");
        verify(projectRepository, times(1)).save(any());
    }
}
```